Technical Design Document

Project: Enterprise Knowledge Search Agent for Kore.ai Documentation and SDK Content

Purpose: This document explains the architecture, trade-offs, chunking strategy, scaling plan, and failure-mode analysis for the candidate assignment described in *Candidate_Assignment.pdf*.

1. Problem and Approach

The goal of this project is to build an enterprise knowledge search assistant that can answer both simple factual questions and more complex research questions across multiple knowledge sources. For this assignment, the primary sources are Kore.ai documentation pages and GitHub README files for SDKs and integrations.

I implemented the system as an **agentic RAG pipeline**. Standard RAG handles ingestion, chunking, embedding, storage, retrieval, and answer generation. Agentic behavior is added on top of that retrieval layer so the system can classify queries, decompose complex questions, call extra tools, and expose a reasoning trace in the UI.

The design intentionally favors transparency over hidden automation. Each major stage is represented explicitly in the backend, and the frontend surfaces not only answers but also source citations, confidence, and the reasoning steps used to produce the answer.

2. Architecture Diagram

```
Data Sources
  |-- Kore.ai documentation pages
  |-- GitHub README / SDK documentation
          |
          v
scraper.py
  -> saves normalized markdown files to backend/data/documents/
          |
          v
loader.py
  -> reads files into Document objects with metadata and source URLs
          |
          v
chunker.py
  -> splits documents into configurable overlapping chunks
          |
          v
embedder.py
  -> creates embeddings with all-MiniLM-L6-v2
```

```
  -> stores vectors + metadata in local ChromaDB
          |
          v
FastAPI /api/query
          |
          v
LangGraph agent
  -> classify query
  -> route to simple RAG / multi-step search / API lookup path
  -> retrieve documentation chunks
  -> optionally call mock API lookup or structured lookup
  -> synthesize grounded answer
  -> compute confidence
          |
          v
React UI
  -> answer display
  -> reasoning trace
  -> source panel with clickable original links
  -> confidence tier and fallback messaging
```

3. End-to-End Data Flow

The application starts at the data-ingestion layer, not at the chat UI. First, documentation pages and GitHub README files are collected and normalized into markdown documents. Each file is then loaded with metadata such as relative source path, source category, and original source URL. Those documents are split into overlapping chunks, embedded, and stored in ChromaDB together with the inherited metadata. This means that by the time a user opens the UI, the system already has a searchable semantic index over the source corpus.

At query time, the end-to-end flow begins in the React frontend. The user enters a question, the UI calls `POST /api/query`, and FastAPI forwards the request into the LangGraph agent. The graph is the core runtime control layer. It does not treat all questions the same. Instead, it uses a classification step to route the query into one of several execution paths:

```
START
  |
  v
classify_query ---- "simple / complex / api_lookup?"
  |
  |-- simple -----> simple_rag ----------------------\
  |                                                   |
  |-- complex ----> decompose_query                   |
  |                   |                               |
  |                   v                               |
  |                 multi_search ---------------------|
  |                                                   |
  \-- api_lookup -> handle_api_lookup ----------------|
```

```
                                                    |
                                                    v
                                            synthesize_answer
                                                    |
                                                    v
                                            score_confidence
                                                    |
                                                    v
                                                   END
```

The reason for this design is efficiency and transparency. Simple factual queries take the shortest path: one retrieval step followed by one generation step. Complex queries are decomposed into smaller sub-questions, each sub-question is searched, and the results are merged before answer generation. API-style questions route through a tool path where a mock API result is collected and combined with documentation evidence. This avoids wasting compute on simple queries while still allowing deeper reasoning for multi-part questions.

The execution path in detail is:

- **classify_query:** uses the LLM to classify the user request as simple, complex, or api_lookup
- **simple_rag:** performs a single vector search and returns the top documentation chunks
- **decompose_query:** breaks a complex query into 2-4 smaller sub-questions
- **multi_search:** retrieves results for each sub-question, deduplicates overlapping chunks, and may augment the result set with structured lookup data for categories like channels, languages, or plans
- **handle_api_lookup:** calls the mock API tool and also retrieves relevant documentation chunks for context
- **synthesize_answer:** builds a grounded prompt using retrieved content and any tool output, then generates the final answer
- **score_confidence:** calculates a retrieval-based confidence score, maps it to high / medium / low confidence, and appends the confidence step to the reasoning trace

Once the graph finishes, FastAPI converts the result into a stable API response that includes the answer, sources, reasoning steps, confidence score, and confidence level. If the score is low, the backend prepends a manual-verification warning. The frontend then renders the answer bubble, confidence tag, source panel, and reasoning panel. In other words, the full application data flow is: source acquisition -> indexing -> query routing -> retrieval/tool use -> grounded synthesis -> confidence scoring -> explanation and citation display in the UI.

4. Component Choices and Trade-offs

Embedding Model: all-MiniLM-L6-v2

I used `all-MiniLM-L6-v2` from Sentence Transformers for embeddings. The main reason was operational simplicity: it is small, fast, inexpensive to run locally, and widely used for semantic search baselines. That made it a good fit for a take-home assignment where fast experimentation mattered.

The trade-off is that this model is optimized for lightweight semantic retrieval, not state-of-the-art enterprise search quality. A larger embedding model or a reranking layer could improve precision, especially for long or highly specific documentation queries. I accepted that trade-off in exchange for speed and easier local execution.

Vector Store: ChromaDB

I chose ChromaDB because it is easy to run locally, supports persistent storage, and works well for a small-to-medium prototype without requiring separate infrastructure. This reduced operational setup and kept the project self-contained.

The trade-off is that ChromaDB is convenient for development, but I would not keep this exact deployment shape for very large production workloads. For the assignment, however, it made the pipeline easy to explain and test.

Agent Orchestration: LangGraph

LangGraph was chosen because the assignment explicitly asks for agentic capability and because graph-based orchestration makes routing and reasoning visible. Instead of hiding all logic in one chain, the system has explicit nodes for query classification, decomposition, search, synthesis, and confidence scoring.

The trade-off is extra complexity compared with a plain retrieval pipeline. However, the graph structure makes debugging easier and supports the assignment requirement to show reasoning and tool usage in the UI.

LLM Strategy: Ollama for local development, Groq-ready abstraction for production

The generation layer is abstracted so the same backend can call either a local Ollama model or a hosted Groq model. This reduces vendor lock-in and supports two operating modes: low-cost local experimentation and faster hosted inference when needed.

The main trade-off is operational reliability. A local Ollama dependency is convenient during development but can become fragile if the local model server is not running. For submission and interview use, a hosted inference path is operationally safer.

5. Chunking Strategy and Rationale

The current chunking configuration is:

- **Chunk size:** 600 characters

- **Chunk overlap:** 100 characters

I chose relatively small chunks to preserve retrieval precision. Documentation queries often ask for a specific setup step, prerequisite, or feature explanation. Smaller chunks reduce the risk of returning a large mixed paragraph where only a small portion is relevant. The overlap is used to avoid losing context at paragraph boundaries.

I also used a recursive splitter with structure-aware separators such as paragraph breaks, line breaks, and markdown headings. This makes chunk boundaries more natural than a fixed-size character chop.

One important observation is that the current settings produce significantly more than the assignment's approximate target of ~1,000 chunks. This was a conscious optimization for retrieval granularity, but it also increases index size and can create more duplicate or near-duplicate evidence. If I had more time, I would run a comparative evaluation using larger chunk sizes such as 1,000-1,200 characters and lower overlap to measure precision, recall, latency, and confidence calibration side by side.

6. Agentic Capability and Tooling

The system supports three execution patterns:

- **Simple RAG path:** one retrieval step followed by answer synthesis for straightforward questions
- **Complex query path:** decompose into sub-questions, retrieve across subtopics, deduplicate, and synthesize
- **API/tool path:** call a mock API lookup and combine its result with documentation context

In addition to vector search, I implemented two extra tools:

- **Mock API lookup** for version, health, bot listing, and rate-limit style queries
- **Structured lookup** for channel/language/plan style tabular data

This satisfies the assignment requirement that the system use at least one additional tool beyond vector search. It also improves explainability because the frontend can show which path and tools were used for an answer.

7. Confidence Scoring and Guardrails

Confidence is based on retrieval evidence rather than model self-reporting. The backend computes confidence as a weighted combination of the best retrieval score and the average of the top three scores. API-backed answers receive a small boost because the tool output is structured and deterministic.

The current confidence tiers are:

- **High:** >= 0.80

- **Medium:** 0.65 to 0.79
- **Low:** < 0.65

For low-confidence answers, the system prepends a fallback warning and shows the supporting sources so the user can manually verify them. Additional guardrails include grounded prompting, explicit instructions not to invent unsupported content, and source citations linking back to the original documentation or GitHub README source.

This is a basic but practical hallucination-reduction strategy. It is intentionally conservative and designed to be explainable during review.

8. Evaluation Attempt and Findings

To evaluate the system more rigorously, I created two markdown-based question banks: one broad evaluation bank and one failure-focused false-positive / false-negative plan. I then built an automated evaluation harness that parses those test banks, runs the questions against the FastAPI backend, captures answers, sources, reasoning traces, confidence values, and tool usage, and scores the responses on correctness, grounding, calibration, routing, and likely failure type.

The evaluation scripts generate detailed JSON, CSV, per-question markdown, summary markdown, and failure-analysis markdown outputs. This turns the project into something closer to an engineering system with measurable behavior instead of a demo-only chatbot.

In the latest usable run, the evaluation covered **143 tests**. The summary outcome was:

- 71 pass-like outcomes
- 68 partial outcomes
- 4 outright failures
- 0 backend errors in that run
- 1 false positive
- 52 false negatives

The strongest category was multi-source reasoning and tool invocation, while the weakest categories were source selection / citation grounding, routing, and architecture-style questions. The dominant failure themes were false negatives, weak retrieval, tool failure, misclassification, confidence miscalibration, and citation drift. This is useful because it shows that the current system is not fundamentally broken, but it is **recall-limited**: it often fails on questions that should be answerable from the indexed corpus.

I view this evaluation attempt as part of the technical design itself. It demonstrates a deliberate effort to test not just whether the app returns text, but whether it is grounded, calibrated, and explainable. If I had more time, I would use these outputs to tune chunking, routing, and retrieval scoring before submission.

9. Scaling to 20M+ Records

To scale this design to tens of millions of records, I would change both the storage layer and the ingestion architecture:

- Replace the local embedded vector store with a distributed managed vector database or search platform that supports sharding, replication, metadata filtering, and operational monitoring
- Separate ingestion into an asynchronous pipeline with queue-based workers for crawl, parse, chunk, embed, and index stages
- Add document versioning and incremental indexing rather than full rebuilds
- Introduce metadata filters for source type, product family, or document freshness to reduce search space before vector ranking
- Add a reranking layer and possibly hybrid lexical + semantic retrieval to improve precision at scale
- Add observability around latency, retrieval hit quality, tool routing frequency, and confidence distribution

At that scale, the architecture becomes more like a search platform than a single-service prototype, but the high-level contract of retrieval + orchestration + grounded answer generation remains the same.

10. Failure Modes and Mitigations

Failure Mode	What Can Go Wrong	Detection	Mitigation
Runtime dependency failure	Backend starts but generation fails because the local LLM server is unavailable	Health checks, startup validation, evaluation harness, backend logs	Use hosted fallback, add startup preflight checks, fail fast with explicit operator guidance
Weak retrieval precision	Semantically related but non-supporting chunks are retrieved	Manual review of retrieved sources, FP/FN evaluation bank, citation drift analysis	Tune chunking, add reranking, introduce hybrid retrieval, deduplicate aggressively
Confidence miscalibration	System is either too cautious on answerable questions or too confident on weak evidence	Confidence calibration test bank and score distribution analysis	Re-tune thresholds, incorporate retrieval diversity, penalize low-overlap evidence
Tool misrouting	API/tool-appropriate questions go through plain retrieval only	Reasoning trace review and routing tests	Improve classifier prompts and add routing heuristics or fallbacks

UI transparency gaps	Users cannot verify where an answer came from	Manual UI inspection and evaluator walkthrough	Expose source URLs, full retrieved chunks, and tool-used reasoning clearly

11. Bonus Features Status

Relative to the optional bonus items in the assignment:

- **LangGraph orchestration:** implemented
- **FP/FN analysis:** implemented through dedicated evaluation banks and an automated evaluation harness
- **Hybrid search:** not yet implemented
- **Conversation memory:** not yet implemented as a complete feature
- **Deployment:** not yet completed

12. Conclusion

This system meets the core shape of the assignment: it ingests enterprise knowledge sources, performs semantic retrieval, adds agentic routing and tool use, shows reasoning and source citations in a React UI, and applies confidence-based fallback behavior. The strongest parts of the implementation are architectural clarity, inspectability, and extensibility.

The main areas I would improve next are operational reliability of the generation layer, retrieval calibration, and a more data-driven comparison of chunking strategies. Even with those limitations, the project demonstrates the intended design thinking: building a grounded enterprise search assistant with explicit orchestration, source traceability, and practical guardrails.

Prepared for the Kore.ai Solutions AI Engineer technical assessment.